OPEN ACCESS
International Journal of Modern Physics C
(2025) 2542003 (20 pages)
© The Author(s)
DOI: 10.1142/S0129183125420033

World Scientific
www.worldscientific.com

Int. J. Mod. Phys. C Downloaded from www.worldscientific.com
by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

An automatic image description generation technology and application for visually impaired individuals

Xilong Qu* and Hu Dong†

School of Information Science & Engineering
Changsha Normal University
Changsha 410100, P. R. China
**quxilong@126.com*
†*32613054@qq.com*

Zhenjin Li‡ and Xiao Tan§

School of Foreign Language
Hunan University of Finance and Economics
Changsha 410205, P. R. China
‡*lizhenjin@hufe.edu.cn*
§*tanxiao@hufe.edu.cn*

Received 26 November 2024
Accepted 6 February 2025
Published 20 March 2025

For visually impaired individuals, the ability to receive real-time textual descriptions of image content can significantly enhance their quality of life and learning. In daily life, automatically generated image descriptions can help them quickly acquire key information, improving their responsiveness and communication experience. In education, particularly in subjects involving complex images or spatial understanding, descriptive content can aid in better comprehension of study materials, thereby narrowing the learning gap between visually impaired students and their peers, and promoting educational equity. This study has developed an automated image description generation system for visually impaired individuals, leveraging deep learning models to accurately diagnose and describe image content. The system employs a combination of Convolutional Neural Networks (CNNs) and Long Short-Term Memory networks (LSTM), integrating attention mechanisms to significantly enhance the quality and performance of the generated descriptions. Trained on large-scale image datasets such as CIFAR-100, Flickr8k and MS COCO, the system can accurately recognize image content and produce corresponding descriptions. Experimental results demonstrate that the proposed system outperforms traditional methods across multiple datasets, generating descriptions with high accuracy and fluency. This paper offers an in-depth description of the system's design and execution, covering the design of the system architecture, dataset selection and preprocessing, model training and

‡Corresponding author.

This is an open access article published by World Scientific Publishing Company. It is distributed under the terms of the Creative Commons Attribution 4.0 (CC BY) License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

optimization and system testing and evaluation. The research results showcase the broad application prospects of automated image description technology in visual aid devices and automated news generation systems, offering valuable references for future technical optimization and application expansion.

Keywords: Automatic image description; deep learning; convolutional neural networks; attention mechanism; assistive technology for the visually impaired.

PACS number: 84.40.UA

Int. J. Mod. Phys. C Downloaded from www.worldscientific.com by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

1. Introduction

1.1. *Research background and significance*

With the rapid advancement of modern technology, the volume of multimedia information has grown exponentially, increasing the demand for automated image description generation. For visually impaired individuals, these descriptions allow them to understand their surroundings, receive important safety alerts, better engage in social activities, enrich their online communication experience, and clearly interpret charts, models and experimental results. Additionally, automated image description technology has wide-ranging applications in fields such as journalism, education, social media and security monitoring. In journalism, automatically generated image descriptions can enhance the efficiency and accuracy of news reporting, enabling visually impaired readers to fully grasp the details of news events. In education, institutions can use this system as a teaching aid to help students better understand image content. In security monitoring, image description generation technology can automatically tag and classify events in surveillance videos, improving response times and processing efficiency.

1.2. *Current research status*

Image description and classification are complex problems that combine computer vision and natural language processing, aiming to convert information from images into human-readable text. In 2015, the encoder-decoder architecture was successfully applied to machine translation, profoundly impacting image description research. This architecture translates images into fixed-length vector representations, which are then converted into language descriptions, achieving remarkable results. Subsequently, the attention mechanism made significant advances in language processing and was incorporated into image description research, significantly improving the accuracy and fluency of descriptions by focusing on key parts of images during generation. Reinforcement learning addressed the issues of vanishing and exploding gradients in deep neural network training, enabling models to more effectively learn the strategy for generating descriptions, further enhancing description quality. The rise of Generative Adversarial Networks (GANs) in image generation also provided new tools for image description research, with researchers exploring the use of GANs to generate more realistic and diverse descriptions.

1.3. *Research objectives*

This study aims to develop an automated image description generation system for visually impaired individuals, leveraging deep learning models to accurately diagnose and generate descriptions of image content, thereby improving their quality of life. By training deep neural network models on large-scale image datasets, the system can accurately recognize image content and produce corresponding descriptions. This paper provides a detailed account of the system's design and implementation, including system architecture design, dataset selection and preprocessing, model training and optimization, and system testing and evaluation. Ultimately, using the CIFAR-100 dataset and evaluation metrics such as the Bilingual Evaluation Understudy (BLEU) algorithm, the quality and performance of the generated image descriptions will be assessed, with a thorough analysis of the experimental results.

2. Technical Background

2.1. *Encoder–decoder architecture*

The encoder–decoder architecture plays a crucial role in generating image descriptions. This framework effectively converts image content into textual descriptions by first transforming images into fixed-length vector representations (encoding process) and then translating these vectors into language descriptions (decoding process). The encoder, typically composed of Convolutional Neural Networks (CNNs), extracts image features and represents them as vectors. The decoder, usually consisting of recurrent neural networks (RNNs) or Long Short-Term Memory networks (LSTMs), converts these vectors into coherent textual descriptions. The success of this architecture largely depends on its ability to handle complex information, especially when establishing associations between images and language. This architecture has achieved remarkable results in image description generation, machine translation and other sequence-to-sequence tasks.

2.2. *Deep learning frameworks*

This system mainly employs TensorFlow and Keras as its frameworks for neural networks and machine learning. Created by Google, TensorFlow is a robust and flexible open-source platform suitable for large-scale numerical computation and deep learning model construction. Its advantages include efficient numerical computation capabilities and extensive application support, providing comprehensive support from model building to training and deployment. Built on TensorFlow, Keras is a high-level neural network API that eases the model creation and training procedures, facilitating more convenient and efficient deep learning model development. The modularity and user-friendliness of Keras allow researchers to quickly construct complex deep learning models and easily debug and optimize them. The combination of TensorFlow and Keras provides strong support for complex deep learning tasks, making the development of an automated image description system feasible.

Int. J. Mod. Phys. C Downloaded from www.worldscientific.com
by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

Int. J. Mod. Phys. C Downloaded from www.worldscientific.com
by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

2.3. *Convolutional neural networks*

CNNs excel at processing visual data. By layering convolutional and pooling operations, CNNs can extract both local and advanced features from images. Convolutional layers apply filters to identify basic elements such as edges and textures, while pooling layers down-sample to decrease data dimensions and computational load, preserving essential information. These features are crucial for applications like identifying images, detecting objects and creating image captions. The use of weight-sharing and local connectivity in CNNs enhances computational efficiency when handling complex visual data, greatly decreasing the number of parameters. For example, the successful deployment of classic models like AlexNet and VGGNet in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) highlights CNNs' formidable capabilities in image analysis tasks. Their success in image recognition provides a solid foundation for developing automated image description systems.

2.4. *Long short-term memory networks*

A specific type of RNN-LSTMs is designed to handle and predict time series data. By leveraging their specialized gate structures (including the input, forget and output) LSTMs are capable of capturing and preserving long-term dependencies in sequences, thereby improving the coherence and precision of generated textual descriptions. In image description generation tasks, LSTMs effectively generate sentences while maintaining contextual semantic coherence. Their successful applications in natural language processing make them a crucial tool for generating image descriptions. For example, LSTMs excel in applications like voice recognition, language translation and generating text, effectively managing the challenges related to prolonged dependency issues in sequential data. Compared to traditional RNNs, LSTMs have a clear advantage in handling long sequences, avoiding problems like vanishing and exploding gradients.

2.5. *Attention mechanism*

The Attention Mechanism significantly improves the performance of image description generation. By focusing on key parts of the image when generating descriptions, the Attention Mechanism allows models to capture important details more accurately, producing more precise and fluent textual descriptions. This mechanism enables models to dynamically allocate different weights to various regions of the input image, highlighting the areas most crucial for generating specific descriptive words. This approach enhances the accuracy and relevance of descriptions, especially when dealing with complex images. The introduction of the Attention Mechanism has improved the encoder–decoder architecture's performance in handling long sequences and complex tasks. The multi-head attention mechanism in the Transformer model has achieved significant success in various language processing applications, establishing itself as a leading state-of-the-art technique today.

Int. J. Mod. Phys. C Downloaded from www.worldscientific.com
by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

3. System Architecture Design

3.1. *Overall system design*

The system comprises four main modules: the image preprocessing module, the model training module, the description generation module and the system integration module. Developed in Python using the Keras framework, these modules work collaboratively to effectively process image data and generate accurate textual descriptions. The image preprocessing module transforms raw image data into a model-suitable format and performs normalization. The model training module leverages deep learning techniques to train on preprocessed image data, enhancing the accuracy of description generation. The description generation module uses the trained model to convert image content into textual descriptions. Finally, the system integration module provides a user interface for image upload, description generation and voice output, facilitating usage by visually impaired individuals.

3.2. *Dataset selection*

To ensure robust performance and broad applicability, the system uses CIFAR-100, Flickr8k and MS COCO datasets for training and testing. The CIFAR-100 dataset includes 32×32 pixel color images in 100 different categories, with 600 images in each category. The Flickr8k and MS COCO datasets offer more diverse and complex images with corresponding descriptions. The combination of these datasets significantly enhances the model's training efficacy and performance.

3.3. *Model design*

Data preprocessing is a crucial step in improving model performance. The CIFAR-100 dataset requires formatting and standardization due to its complex content structure. Functions such as load_descriptions and write_descriptions are defined to read, process and write the description data into new files, as illustrated in Fig. 1.

(1) Load Image Dataset: Sequentially load image files into memory.
(2) Convert Images to Numpy Arrays: Facilitate subsequent processing.

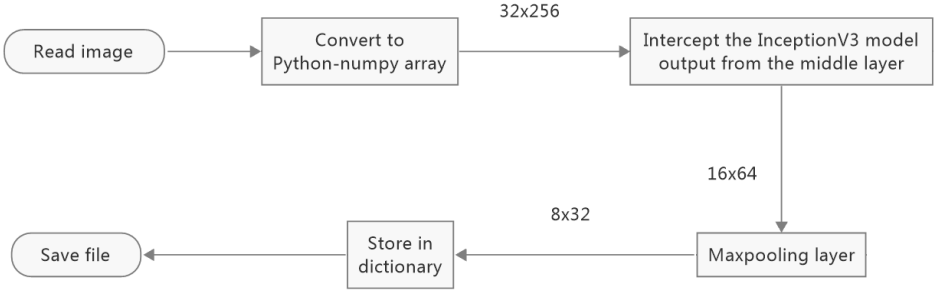


Fig. 1. Image preprocessing module workflow.

(3) Normalize Images: Preprocess using the InceptionV3 model to meet model requirements.
(4) Extract Features: Input preprocessed images into the pretrained model to obtain output feature representations.
(5) Store Features: Save output features in a dictionary with filenames as keys and feature values.
(6) Save Data in Batches: Use the dump() function to save dictionary data in batches of 100.

To create a varied set of training samples, we use data augmentation methods including rotating, scaling, translating and flipping images. This approach not only increases the training dataset but also enhances the model's robustness, enabling it to better manage various image inputs. The code snippet below illustrates data augmentation:

Code 1: Data Augmentation

```
from keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing import image
import numpy as np
import matplotlib.pyplot as plt

# Initialize ImageDataGenerator with augmentation options
augmentor=ImageDataGenerator(
    rotation_range=25,          # Rotate images within 25 degrees
    width_shift_range=0.15,     # Shift width by 15%
    height_shift_range=0.15,    # Shift height by 15%
    shear_range=0.15,           # Shear by 15%
    zoom_range=0.15,            # Zoom in/out by 15%
    horizontal_flip=True,       # Flip images horizontally
    fill_mode = 'reflect'       # Reflect edges for new pixels
)

# Load and preprocess an image
img_path='path/to/your/image.jpg'
loaded_img=image.load_img(img_path)                 # Load image
img_array=image.img_to_array(loaded_img)            # Convert to array
img_array=np.expand_dims(img_array, axis = 0)       # Expand dims

# Generate and display augmented images
augmented=augmentor.flow (img_array, batch_size=1)
for idx, new_img in enumerate(augmented):
    if idx >=4: # Display 4 augmented images
        break
    plt.figure (idx)
    plt.imshow (image.array_to_img(new_img[0]))
    plt.axis ('off')

    plt.show()
```


Int. J. Mod. Phys. C Downloaded from www.worldscientific.com by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

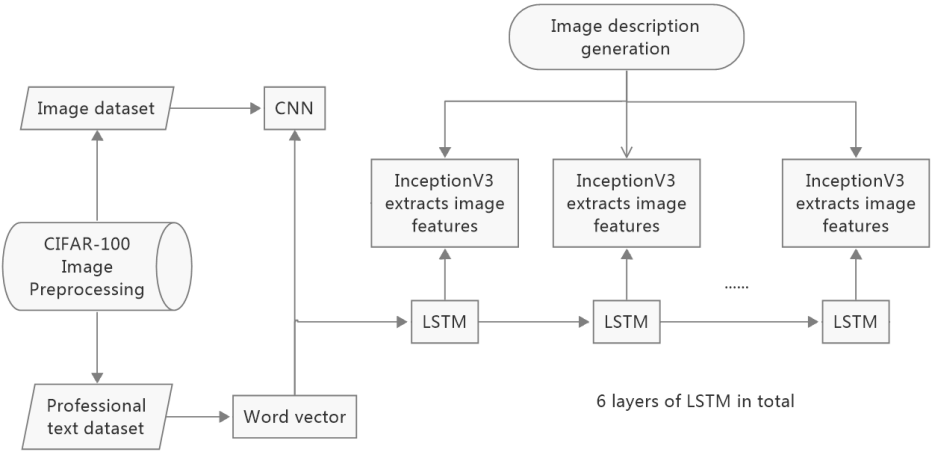


Fig. 2. CNN–LSTM model data framework.

By incorporating these preprocessing and augmentation techniques, the system effectively enhances the training dataset's diversity and robustness, ultimately improving the model's ability to generate accurate and coherent image descriptions.

3.4. *Model construction*

In image description tasks, the mainstream design employs an encoder–decoder structure, as illustrated in Fig. 2. The encoder is responsible for encoding the input image into feature tensors, while the decoder decodes these feature tensors into word probabilities. The encoder's task is completed during image preprocessing, where the image is transformed into feature tensors.

The model incorporates an attention mechanism, enabling it to concentrate on particular aspects of the features during each time step. Attention is represented as a probability weight vector, which is multiplied by the feature tensor and used as input to the LSTM. The decoder part is primarily an LSTM network, a variant of the RNN, which excels at handling long sequence inputs. The feature tensor processed by the attention mechanism is combined with the LSTM network's previous time step output and used as input for the next step. The LSTM's output gets transformed through a dense layer into a vector that corresponds to the length of the vocabulary, with values representing the probability of each word. The weights αt are derived from the LSTM's output at the previous timestamp, as described by the following equations:

$$\begin{cases} z_t = \alpha_t^T * a, \\ e_z = \text{relu}(f_{e1} * s_e), \\ \alpha_t = \text{softmax}(f_c(e_t)). \end{cases}$$

Int. J. Mod. Phys. C Downloaded from www.worldscientific.com by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

The model is primarily built using the Keras framework and consists of the following components:

(1) Feature Extraction (FE): A dense layer with ReLU activation, followed by a reshape operation, alters the input feature dimensions from $4 \times 4 \times 384$ to 32×256.
(2) Sequence Embedding (SE): Embeds the current time step's words into a 32×256 vector.
(3) Feature Transformation (FT): Transforms fe through a fully connected layer into 32×256 dimensions, which is then fused with se in the attention mechanism. The result is activated by ReLU and transformed by a fully connected layer with softmax into a 32×64 weight tensor.
(4) Attention-Weighted Features: The weight tensor is multiplied by the feature tensor to generate attention-weighted image features, with dimensions 32×256.
(5) Decoder: The image features weighted by attention are merged with SE and then fed into the LSTM. The LSTM's output is converted through a dense layer with softmax activation into a word probability vector, which serves as the model's output.

3.5. *Model training and optimization*

Training is performed on large-scale datasets, optimizing performance by fine-tuning hyperparameters and applying techniques such as Dropout to avoid overfitting. During training, the system employs the cross-entropy loss function along with the Adam optimizer, adjusting the learning rate as well as other hyperparameters based on the validation set performance to achieve optimal results on both the training and validation sets. The training steps are as follows:

(1) Load Data: Load training and validation datasets, including image data and corresponding textual descriptions.
(2) Encode Descriptions: Encode and pad the descriptions to make them suitable for model input.
(3) Data Augmentation: Employ methods like rotating images, scaling, translation and flipping to create a variety of training samples.
(4) Compute Loss: Apply the cross-entropy loss function to evaluate the discrepancy between the model's outputs and the actual descriptions, optimizing the model parameters using gradient descent.
(5) Prevent Overfitting: Add Dropout layers during training to enhance the model's generalization ability.

The model utilizes categorical cross-entropy for the loss function and an adaptive optimizer. The CIFAR-100 dataset is already split into training (2000 samples), validation (1000 samples) and test sets (1000 samples). The training and validation sets are loaded together with their associated descriptions and feature data. The vocabulary is created, and the vocabulary size and the maximum description length are determined for training.

Int. J. Mod. Phys. C Downloaded from www.worldscientific.com
by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

Code 2: Dropout

```
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, LSTM, Embedding,
  TimeDistributed
from keras.optimizers import Adam
def build_cnn_lstm_model(vocab_size, seq_length):
  # Create a sequential model
  model=Sequential ()
  # Convolutional layer + max pooling + dropout
  model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
  model.add(MaxPooling2D((2, 2)))
  model.add(Dropout(0.25))
  # Second convolutional layer + max pooling + dropout
  model.add(Conv2D(64, (3, 3), activation='relu'))
  model.add(MaxPooling2D((2, 2)))
  model.add(Dropout(0.25))
  # Third convolutional layer + max pooling + dropout
  model.add(Conv2D(128, (3, 3), activation='relu'))
  model.add(MaxPooling2D((2, 2)))
  model.add(Dropout(0.25))
  # Flatten output and add dense layer with dropout
  model.add(Flatten())
  model.add(Dense(256, activation='relu'))
  model.add(Dropout(0.5))
  # Embedding layer + LSTM + TimeDistributed dense layer
  model.add(Embedding(vocab_size, 256, input_length=seq_length))
  model.add(LSTM(256, return_sequences=True))
  model.add(TimeDistributed(Dense(vocab_size, activation='softmax')))
  # Compile the model with loss and optimizer
  model.compile (loss='categorical_crossentropy', optimizer=Adam(), metrics=['accuracy'])
  return model
```

Through these structured steps and methodologies, the system is trained to generate accurate and coherent image descriptions, effectively improving the quality of life and learning experiences for visually impaired individuals.

3.6. *System integration and deployment*

After system integration, images are input through real-time capture or triggered snapshots. The system automatically generates corresponding textual descriptions and outputs them via speech synthesis technology, making it convenient for visually impaired individuals to use. The description generation process includes mapping prediction results to corresponding category names, interpreting probability distributions, or matching model outputs with predefined descriptions. The model analyzes image content, extracts features and makes predictions, outputting a probability distribution or category label that represents the model's understanding and judgment of the image content. These prediction results are then converted into coherent natural language sentences or paragraphs and returned to the user.

Int. J. Mod. Phys. C Downloaded from www.worldscientific.com
by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

Int. J. Mod. Phys. C Downloaded from www.worldscientific.com
by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

To ensure ease of understanding, the module determines the maximum length of the description sentences at startup and loads a carefully trained model.

Using the provided description files, the system constructs an accurate vocabulary. Utilizing the Tkinter library, the module implements a file selection feature, allowing users to upload images easily. After an image is chosen, the InceptionV3 model begins extracting characteristics from the image, which are subsequently input into the model to produce prediction results. This result, a word probability distribution vector, is converted into a complete descriptive sentence, displayed in real-time on the user interface and simultaneously broadcast via speech synthesis. The system workflow is illustrated in Fig. 3.

Upon running the program, the system can process images either automatically or manually. In manual mode, users click the "Select Image" button to see a file selection window, where they can easily find and select the JPEG images they want to upload. If the user chooses to exit, the system returns to its initial state. When the program continues to run, users can open the file selection window by clicking the "Select Image" button or exit the program by clicking the "Exit" button. In the file selection window, users can see their chosen JPEG images. Clicking the cancel button closes the window and returns to the prompt window, as shown in Fig. 4.

After users select an image and submit it to the system, a prompt dialog box appears. At this point, the main program thread creates a child thread to handle the description generation computation task. This multithreaded approach allows the

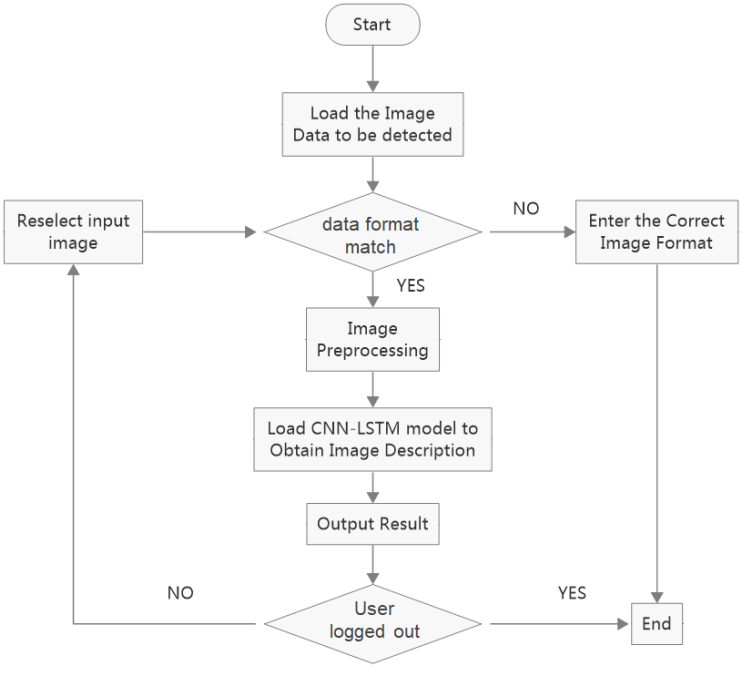


Fig. 3. Image description system workflow.

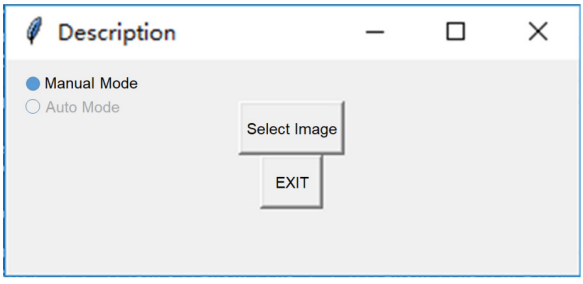


Fig. 4. Prompt window for image selection.

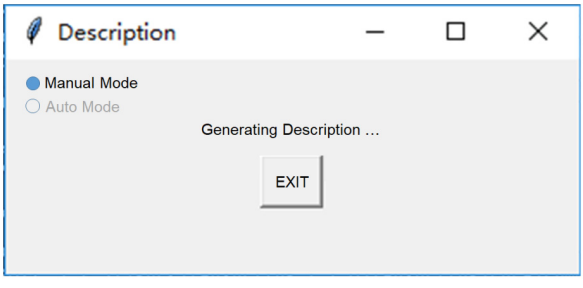


Fig. 5. Prompt window notification.

system to remain responsive during description generation, avoiding interface freezes or delays, and ensuring stable and efficient program operation. Users can choose to close the image display window or keep it for reference without causing any interference or interruption. If users choose to stop the current operation, they can end the program by pressing the close button at the top right corner of the window or by choosing the "Exit" button (Fig. 5).

Once the image description generation process is complete, users see a prompt window displaying the system-generated image description. In automatic mode, the system reads the generated text information to the user.

4. System Testing

4.1. *Experimental setup*

The hardware environment for this experiment is detailed in Table 1.

Model training and testing were conducted in this environment using the Python programming language and the Keras framework for constructing and optimizing the deep learning model.

4.2. *Functional testing*

The loss values post-training reveal an overfitting problem, with the validation set loss exceeding the training set loss. This indicates that the model has excessively

Int. J. Mod. Phys. C Downloaded from www.worldscientific.com
by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

Table 1. Experimental environment.

Operating system	CPU	Memory	GPU	Hard disk
Microsoft Win 11 (64-bit)	Intel(R) Core(TM) i5-6300HQ @ 2.30 GHz	16.0 GB	Intel(R) HD Graphics 530	WDC WDS500G2B0C-00PXH0

learned the particular patterns and noise of the training set, resulting in subpar performance on the validation set. Figure 6 shows training images, with the rightmost column displaying the nearest training examples to its left column.

We evaluated the model's training and validation performance on the CIFAR-100 dataset using the cross-entropy loss to ensure high-quality descriptions. This loss function, commonly applied in classification tasks, measures the divergence between the predicted probabilities and the actual label distributions. Higher scores on this metric indicate that the generated image descriptions are closer to the human-annotated descriptions in the dataset, signifying better performance. The training loss curve is shown in Fig. 7.

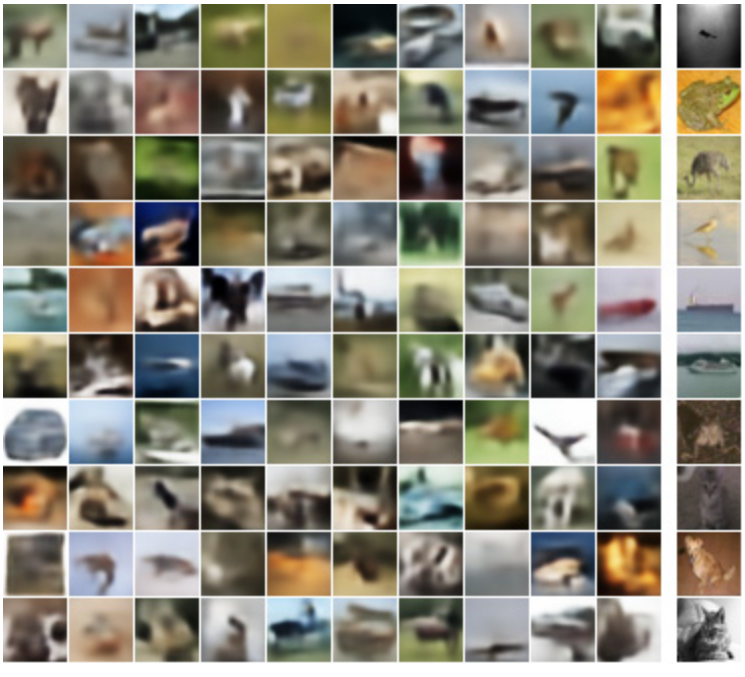
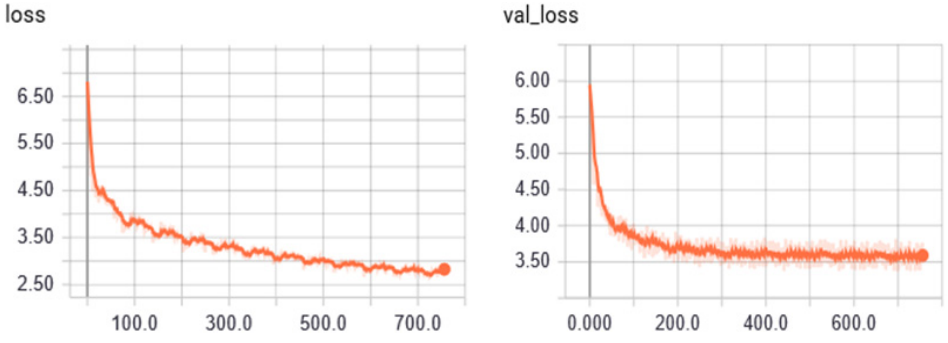


Fig. 6. Training images.

Int. J. Mod. Phys. C Downloaded from www.worldscientific.com
by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.



(a) Loss on training set (b) Loss on validation set

Fig. 7. Training and validation loss curves on CIFAR-100 dataset.

The horizontal axis (iterations) represents the number of training epochs, each epoch denoting one complete pass over all training samples, including forward and backward propagation. The vertical axis (loss value) represents the model's loss per iteration, highlighting the gap between predictions and actual labels. Lower values denote more accurate predictions. The loss decreases rapidly in the initial training stages, slows in the mid-stages and further slows in the later stages.

To assess the quality of the generated descriptions the BLEU score is used as the primary metric. The BLEU score measures the accuracy and fluency of descriptions by calculating the n-gram match between the generated and reference descriptions.

4.3. *Experimental results*

The experimental results show that the proposed system surpasses traditional methods across various datasets. Specifically, on the CIFAR-100 dataset, the BLEU scores are as follows: BLEU-1: 0.726, BLEU-2: 0.324, BLEU-3: 0.633, BLEU-4: 0.297, indicating high accuracy and fluency of the generated descriptions. In comparison, the traditional CNN–CNN model scores lower on the same dataset, further proving the superiority of the proposed CNN–LSTM model.

4.4. *Model comparison analysis*

In image description generation, comparative experiments between the CNN–CNN and CNN–LSTM frameworks reveal that the CNN–LSTM framework performs better in generating descriptions. The experiments show that with a fixed network depth of six layers, the CNN–LSTM framework significantly outperforms the CNN–CNN framework on the BLEU metric.

The strength of the CNN–LSTM framework is its use of LSTM networks as decoders, which efficiently process sequential data and preserve the relational

Int. J. Mod. Phys. C Downloaded from www.worldscientific.com
by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

Table 2. BLEU scores for CNN–LSTM model.

Standard	Score
BLEU-1	0.726
BLEU-2	0.324
BLEU-3	0.633
BLEU-4	0.297

Table 3. BLEU scores for CNN-CNN model.

Standard	Score
BLEU-1	0.602
BLEU-2	0.531
BLEU-3	0.397
BLEU-4	0.117

Int. J. Mod. Phys. C Downloaded from www.worldscientific.com by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

information between words, resulting in more coherent and accurate descriptions. Although the CNN–CNN framework can describe rich image content in some cases, its description quality and accuracy are slightly inferior to the CNN–LSTM framework.

The BLEU score serves as the primary evaluation metric in this paper, measuring the n-gram match between generated and reference descriptions. As shown in Table 2, the CNN–LSTM framework maintains a high BLEU-1 score of 0.726, confirming its superiority in image description generation.

The BLEU scores for the CNN–CNN model are slightly lower than those of the CNN–LSTM framework, as shown in Table 3.

During the testing phase, a sequential word generation strategy is employed, meaning one word is generated at each time step and passed to the next step as input. While CNNs excel in image processing, they may not perform as well as LSTMs in handling sequential data. CNNs are typically used for local feature recognition, whereas LSTMs specialize in sequence data processing, making them more suitable for temporal modeling.

In terms of performance, apart from parameter loading time, the testing speeds of both models are nearly identical, averaging 0.4 s to generate descriptions for a batch of 100 images. The CNN–LSTM model's superior temporal information processing and effective long-distance dependency modeling highlight its advantages and potential applications in image description generation. Table 4 displays the results of the CNN–LSTM and CNN–CNN models.

Table 4. Results of CNN–LSTM and CNN–CNN models.

Pic	CNN–LSTM	CNN–CNN
	A living room with a table and chairs	A wooden table topped with lots of wooden furniture
	A bathroom with a toilet sink and shower	A bathroom with a sink toilet and shower
	A large white airplane sitting on top of an airport tarmac	A group of plane sitting on top of an air tarmac
	A bathroom with a sink and mirror	A kitchen with a sink and window
	Man; suit; tie; glasses	Man; suit; tie

Int. J. Mod. Phys. C Downloaded from www.worldscientific.com
by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

4.5. *Implementation of visual aids*

This study successfully integrated the system into a visual aid device to help visually impaired individuals understand their surroundings in real time. The device captures images through a camera, uses the model to generate image descriptions, and feeds the descriptions back to the user via speech synthesis technology.

We wrote the image capture and processing program in Python, integrating the camera with the model, and used the pyttsx3 library for speech synthesis, converting

the generated descriptions into speech output. The device was tested in various environments (indoor, outdoor, well-lit and dimly lit) and performed reliably, with an average total time from image capture to speech output within one second, meeting real-time requirements. The code is as follows:

Code 3: Image Recognition and Speech Output

```
import cv2
import pyttsx3
def capture_image():
    cap=cv2.VideoCapture(0)
    ret, frame=cap.read()
    cap.release()
    return frame
def describe_image(image):
    preprocessed_image=preprocess_image(image)
    description=generate_desc (model, tokenizer, preprocessed_image,
  max_length)
    return description
def speak_description(description):
    engine=pyttsx3.init()
    engine.say(description)
    engine.runAndWait()
while True:
    image=capture_image()
    description=describe_image (image)
    speak_description(description)
```

4.6. *Implementation of automated news generation*

The system is also applied in automated news generation by analyzing news images, generating descriptions and automating the process of image upload, description generation and news publication via API. Test results show high accuracy in descriptions for different types of news images, significantly improving the efficiency and accuracy of news reporting. The code is as follows:

Code 4: News Automation

```
import requests
def fetch_news_images(api_url):
    response=requests.get(api_url)
    return response.json()
```


Int. J. Mod. Phys. C Downloaded from www.worldscientific.com
by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

(*Continued*)

Code 4: News Automation

```
def generate_news_description(image):
    preprocessed_image=preprocess_image(image)
    description=generate_desc(model, tokenizer, preprocessed_image,
  max_length)
    return description
def post_news(api_url, description):
    payload={'description': description}
    response=requests.post(api_url, data=payload)
    return response.status_code
api_url='https://newsapi.example.com/images'
news_images=fetch_news_images(api_url)
for image in news_images:
    description=generate_news_description(image)
    status=post_news ('https://newsapi.example.com/post', description)
    if status==200:
        print ('News posted successfully!')
    else:
        print('Failed to post news.')
```

Testing confirmed high accuracy levels in the descriptions for various types of news images.

5. Conclusion and Future Work

5.1. *Main conclusions*

This study successfully developed an automated image description generation system tailored for visually impaired individuals, employing advanced deep learning techniques to achieve precise image content diagnosis and natural language description generation. By integrating an architecture that combines CNNs and LSTMs with an attention mechanism, the system demonstrated superior performance across multiple datasets, including CIFAR-100, Flickr8k and MS COCO. The generated descriptions were both accurate and fluent, surpassing traditional methods. This system not only provides a practical assistive tool for visually impaired individuals but also showcases its broad application potential in fields such as education, journalism and security monitoring.

5.2. *Research contributions*

This study made significant advances in image description generation technology, providing robust technical support for visually impaired individuals.

Int. J. Mod. Phys. C Downloaded from www.worldscientific.com
by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

(1) Architectural Innovation: The research designed a model architecture combining CNN and LSTM with an attention mechanism, significantly enhancing the quality of image description generation. The CNN extracts image features, the LSTM generates coherent textual descriptions and the model focuses on crucial image areas by utilizing the attention mechanism during description generation.
(2) System Design and Implementation: The system adopts a modular design, encompassing image preprocessing, model training, description generation and system integration. Using the TensorFlow and Keras frameworks, the system achieves complex functionality while ensuring convenient and efficient development.
(3) Practical Application Validation: Application testing in visual aid devices and automated news generation systems demonstrated that the generated descriptions are accurate and easy to understand. Specifically, for visually impaired individuals, the system can provide real-time feedback on image content through speech synthesis technology, significantly enhancing users' quality of life and social engagement.

5.3. *Future work*

Future research and development will focus on the following areas to further optimize and expand the technology of automated image description generation:

(1) Model Optimization and Improvement: Continuous optimization of the CNN–LSTM model architecture will be pursued, exploring more efficient parameter settings and network structure designs. Systematic performance metric analysis and loss function adjustments will be conducted to further enhance the model's description generation capability and accuracy. Additionally, the incorporation of the latest deep learning techniques, such as the Transformer architecture, will enhance the model's capacity to manage intricate images and lengthy textual descriptions.
(2) Dataset Expansion and Diversity Enhancement: Efforts will be made to explore and integrate a richer variety of image datasets, including images from different scenes, themes and styles, to enhance the model's ability to generalize and provide comprehensive descriptions. For example, including more real-world complex scene images will enhance the model's performance in handling diverse images.
(3) Exploration of Practical Application Scenarios: The research findings will be applied to broader practical production and application fields, exploring the potential of image description generation technology in intelligent image recognition, automatic image annotation and intelligent recommendation systems. Particular emphasis will be placed on evaluating the model's effectiveness and performance in high-demand areas such as medical image analysis, autonomous driving and robotic vision systems.

Int. J. Mod. Phys. C Downloaded from www.worldscientific.com
by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

(4) Algorithm Fusion and Deep Integration: Research will be conducted on integrating other cutting-edge deep learning algorithms and technologies, such as GANs and more advanced attention mechanisms, with the existing model. Attempts will be made to achieve deep integration to further enhance the system's intelligence and generation capabilities. The adoption of multi-modal learning methods, combining information from images, text and sound, will be considered to improve the system's overall performance.

(5) User Experience Enhancement: Based on user feedback, the system interface design and interaction methods will be optimized to enhance user experience. Specific measures will include simplifying user operation processes, improving response speed and enhancing the naturalness and accuracy of voice feedback, ensuring the system is more convenient and practical, and meeting the needs of visually impaired individuals and other user groups. Additionally, personalized setting functions will be developed, allowing users to adjust various system parameters and functions according to their preferences.

Acknowledgments

This research was supported by the Scientific Research Key Project of Hunan Education Department (Nos. 24A0791, 24B0926, 24A0786 and 21A0592). Changsha Municipal Natural Science Foundation (kq2014063). Provincial Natural Science Foundation of Hunan (2023JJ30101 and 2025JJ70437. Ministry of Education Humanities and Social Sciences Research Youth Fund Project (22YJCZH025).

ORCID

Hu Dong https://orcid.org/0000-0002-8013-9429
Zhenjin Li https://orcid.org/0000-0001-5772-8359
Xiao Tan https://orcid.org/0000-0002-2124-2493

References

1. R. Bernardi, *J. Artif. Intell. Res.* **55**, 409 (2016).
2. S. S. P. R. Amma, *Adv. Sci. Technol. Eng. Syst. J.* **7**, 119 (2022).
3. V. Verma, *Int. Conf. Mach. Learn. Image Process. Netw. Secur. Data Sci.* **1763**, 82 (2022).
4. Y. Tang, *Proc. 6th Int. Conf. Imaging Signal Process. Commun.* **6**, 10 (2022).
5. Z. Hossain, *ACM Comput. Surv.* **51**, 1 (2019).
6. C. Ningthoujam, *Contemp. Issues Commun. Cloud Big Data Anal.: Proc. CCB* **229**, 240 (2022).
7. F. Huang, *Mach. Learn.* **109**, 2313 (2020).
8. T. Jiang, *Vis. Comput.* **35**, 1655 (2019).
9. A. Karpathy, *IEEE Trans. Pattern Anal. Mach. Intell.* **37**, 3128 (2015).
10. H. Katpally, *Proc. IEEE Int. Conf. Semantic Computing* (Institute of Electrical and Electronics Engineers Inc., 2020), p. 61.

Int. J. Mod. Phys. C Downloaded from www.worldscientific.com
by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.

11. M. J. Khan, *CIKM* **2699**, 1 (2020).
12. R. Kiros, *Int. Conf. Mach. Learn.* **595**, 603 (2014).
13. H. Ma, *Math.* **12**, 286 (2024).
14. S. Tong, *Adv. Neural Inf. Process. Syst.* **36**, 1 (2024).
15. Q. Li, *Inf. Fusion* **102**, 102037 (2024).
16. G. Kulkarni, *IEEE Trans. Pattern Anal. Mach. Intell.* **35**, 2891 (2013).
17. P. Kuznetsova, *Trans. Assoc. Comput. Linguist.* **2**, 351 (2014).
18. L. Xiaoxiao, *Vis. Comput.* **35**, 445 (2019).
19. J. Mao, *arXiv* **1**, 17 (2015).
20. A. Nursikuwagus, *J. Imaging* **8**, 294 (2022).
21. Z. Wang, *Neurocomputing* **587**, 127674 (2024).
22. V. Ordonez, *Int. J. Comput. Vis.* **119**, 46 (2016).
23. J. Chen, *Phys. Med. Biol.* **68**, 05TR01 (2023).
24. S. P. Manay, *Emerg. Res. Comput. Inf. Commun. Appl.: ERCICA* **1**, 511 (2020).
25. K. M. Safiya, *Multimed. Tools Appl.* **83**, 59413 (2024).
26. D. Louzada Fernande, *arXiv e-prints* **2202**, 05331 (2022).
27. R. Malhotra, *Multimed. Tools Appl.* **82**, 44977 (2023).
28. D. Gurari, *Comput. Vis.–ECCV 2020: 16th Eur. Conf.* **417**, 434 (2020).
29. S. R. Sreela, *Information* **10**, 354 (2019).
30. H. J. A. Sharma, *Mod. Phys. Lett. B* **34**, 2050315 (2020).
31. D. H. Fudholi, *IAES Int. J. Artif. Intell.* **13**, 23 (2024).

Int. J. Mod. Phys. C Downloaded from www.worldscientific.com
by 115.70.50.33 on 09/19/25. Re-use and distribution is strictly not permitted, except for Open Access articles.